Проект

Вычислитель отличий

Начат: 28.05.2024

Проверяющий: **Дмитрий Полковников**
Студент: **Олег Мартынов**
https://github.com/MartinMeer/java-project-71

Демонстрационный проект:
https://asciinema.org/a/NFlQgLVMu1ymFsqg4ESeOQeXi

Вывод справки

Первым шагом в этом проекте нужно вывести справочную информацию по нашей утилите:

```
# Порядок вывода параметров не важен
./build/install/app/bin/app -h

Usage: gendiff [-hV]
Compares two configuration files and shows a difference.
  -h, --help      Show this help message and exit.
  -V, --version   Print version information and exit.
```

Этот хелп сгенерирован автоматически с помощью специальной библиотеки, упрощающей создание утилит командной строки (cli). В примере использована одна из популярных библиотек в java, которая называется *picocli*. И в этом проекте мы будем активно её использовать.

Консольная утилита должна будет использовать опции и параметры. Для этого библиотека *picocli* предлагает использование аннотаций (`@Option`, `@Parameters`, `@Command`). С механизмом работы аннотаций более подробно вы познакомитесь позже, а на данном этапе воспринимайте их как вызов метода, в который можно передать нужные значения.

Оформление пакета

Любая программа, распространяемая в виде пакетов, одновременно является и библиотекой. Другими словами, библиотеку которую мы разрабатываем, можно установить как зависимость в любом другом пакете. Пример того, как другие пакеты могли бы использовать наш *gendiff*:

```
import hexlet.code.Differ;

var diff = Differ.generate(filePath1, filePath2);
System.out.println(diff);
```

То есть ваш проект должен предоставлять класс `Differ` в пакете `hexlet.code` со статическим методом `generate()`, вызов которого возвращает разницу между данными в виде строки. А эти данные уже используются где-то в другом месте. Например, в нашей cli-утилите

Ссылки

- CLI Builder (picocli)
- Полностью настроенный проект Gradle — пример, в котором можно подсматривать настройку окружения.
- Объяснятель шелл команд
- Скрипты, модули и библиотеки
- Используем make — новый взгляд на старый инструмент
- Что такое Makefile и как начать его использовать
- Руководство по современному make

Задачи

1. Убедитесь, что у вас в системе установлена утилита *make*, выполнив в терминале команду `make -v`
2. Создайте в директории проекта *Makefile* с командой `run-dist`, которая выполняет запуск исполняемого файла:

```
./build/install/app/bin/app
```

Теперь достаточно в терминале набрать `make run-dist` чтобы запустить исполняемый файл полученный в результате сборки, вместо набора длинной команды.

3. Изучите [пример готового приложения в документации picocli](#) и опишите нужную логику в файле *src/main/java/hexlet/code/App.java*, отвечающего за работу консольного приложения

4. Выполните сборку проекта при помощи gradle задачи `installDist` и запустите исполняемый файл *app* с опцией *-h*. Убедитесь, что справка отображается как в примере выше

Вопросы и ответы

Ограничено ли время выполнения проекта?

Нет. Если вы учитесь самостоятельно или с персональным наставником, проходить проект можно в удобном для себя темпе. Независимо от формата обучения, лучше поддерживать постоянный прогресс и тратить на выполнение проекта (без учёта ревью) до 2-4 недель. Если вы учитесь в группе, этот срок поможет уложиться в сроки большинства учебных программ

Что делать, если возникли трудности?

Выполнение проекта сопряжено с большим количеством сложностей и некоторые шаги даются тяжело. Испытывать трудности нормально, но не стоит зависать над решением часами. Объяснить задачу другими словами, дать подсказку или указать явно на ошибку может ваш наставник. Обратитесь в чат своей группы или к персональному наставнику, если уже больше часа не можете продвинуться в решении

В процессе выполнения опирайтесь на наши [советы по прохождению проектов](#), особенно если занимаетесь без наставника

Когда проверяется проект?

И автоматизированные тесты, и наставник проверяют проект в конце, когда завершён последний шаг. При этом обращаться за поддержкой можно на каждом шаге

Как долго наставник проверяет проект?

Обычно — до 3 рабочих дней. Наставник предупредит, если не будет успевать. Если наставник опаздывает со сроками ответов или вас не устраивает формат ответов, сообщите куратору — он поможет разрешить спорные ситуации

Завершено

Обсуждение

Вопросы других участников | Мои вопросы

Проект «Вычислитель отличий / Java-разработчик»
↳ Контрольная точка «step 3»

Сали Абдуллаева тс 1 день назад

я не понимаю что требуется в этом шаге - это если коротко

если подробнее, то с мейкфайлом - понятно, разобралась.

еще частично понятно, что такое picocli и что надо запустить исполняемый файл app с опцией -h.

Но остальное, что делать и как хз, даже с ссылками на доп.инфу и чтением комментариев мне непонятно. Не вижу конкретное предложение - создайте у себя то-то ,чтоб было так-то.. Пожалуйста, обьясните, что надо сделать. Что это за оформление пакета и для чего и вообще при чем тут все и тд?

Ответить

Показать ещё комментарии (2)

Maksim Litvinov Поддержка около 18 часов назад

Верно, посмотрите на пример работы в шаге. Это команда и то, что она выводит

Ответить

Новый комментарий...

Проект «Вычислитель отличий»
↳ Контрольная точка «step 3»

Баир Цыдендамбаев ТС Топик закрыт
01 февраля 2024

Вдруг кому поможет: https://picocli.info/#_help

1

Баир Цыдендамбаев ТС 01 февраля 2024

Пока такое впечатление, что это не picocli, а Boku no Pico какое-то. Немного не очевидная документация, мягко говоря.

5

Проект «Вычислитель отличий»
↳ Контрольная точка «step 3»

Иван Дёгтев ТС Топик закрыт 08 января 2024

Чтож, чувствую, сей проект будет непростым, так что буду оставлять некоторые заметки, мб для собственного закрепления или для след. участников. Вспоминаются серии DS с записями перед боссами, лол. Чтож, я пару часов тупил,

ведь утилита picocli новая и хз как вообще писать. Что в итоге сделал - закопипастил себе этот пример(самый первый), он вроде легкий, понятный и стал удалять все ненужное, тестируя, что получается в итоге. Как я понял, два окна с версией и помощью (-V, -h) у утилиты дефолтные и включаются тем, что прописываешь в начале перед классом в `@Command mixinStandardHelpOptions = true` Далее я все снес ненужное из примера - типа опций и параметров (хз.

развернуть

8

Показать ещё комментарии (2)

Maksim Litvinov **Поддержка** 22 января 2024

Ребята, это проект и это его фишка. Тут все так, как будет в реальной разработке. Нужно читать документацию, самостоятельно разбираться с новыми библиотеками. Вообщем, полное погружение, как на работе

2

Проект «Вычислитель отличий / Java-разработчик»
↳ Контрольная точка «step 3»

Александр тс Топик закрыт 08 декабря 2023

При запуске make run-dist пишется "make: *** Нет правила для сборки цели «run-dust». Останов." Текст Makefile

Makefile

run-dist: ./build/install/app/bin/app Отдельно ./build/install/app/bin/app выполняется. Вроде всё так же как и в первом проекте. Версия утилиты GNU Make 4.3

Nikolai Gagarinov **Поддержка** 08 декабря 2023

Добрый день.

Убедитесь, что в Makefile вы используете табы, а не пробелы. Загляните в наш эталоне - https://github.com/hexlet-boilerplates/java-package/blob/main/Makefile

Проект «Вычислитель отличий / Java-разработчик»
↳ Контрольная точка «step 3»

Павел Васильев тс Топик закрыт 15 марта 2023

Здравствуйте! Всё время задаюсь вопросом.. Подскажите, когда каждый раз говорится об "исполняемом файле", что имеется ввиду, файл с байт кодом или с машинным кодом?

Решено

Показать ещё комментарии (1)

Nikolai Gagarinov **Поддержка** 16 марта 2023

Разница в том, что другие файлы могут содержать приложение, модули, библиотеки. Они внутри себя содержат только определения (тело методов) и не запускаются сами по себе, а подключаются (например в исполняемые файлы).

Отмечено как решение

Проект «Вычислитель отличий»
↳ Контрольная точка «step 3»

Lilia Litvinenko тс Топик закрыт 26 января 2023

> Изучите пример готового приложения в документации picocli и опишите нужную логику в файле src/main/java/hexlet/code/App.java, отвечающего за работу консольного приложения

можно понятнее объяснить, что нужно делать? какую логику описать и откуда ее брать? ничего не понятно на этом шаге

12

Maksim Litvinov Поддержка 27 января 2023

Нужно сделать так, чтобы при запуске исполняемого файла выводилась справочная информация по нашей утилите:

```
▶ Code (possible spoiler)
```

Такой вывод помогает получить утлита picocli, которая облегчает создание утилит командной строки. Посмотрите в документации пример, как с ней работать. И сделайте по аналогии, чтобы при запуске исполняемого файла мы получали такой вывод

Проект «Вычислитель отличий»
↳ Контрольная точка «step 3»

Arcady тс Топик закрыт 18 сентября 2021

Здравствуйте! Сделал, всё работает, как велено, на данном шаге. Но не понял, к чему сейчас абзац **Оформление пакета** в условии... Сейчас у меня в папке `src/main/java/hexlet/code/` находятся 2 файла - `App.java` и `CommandLine.java` . А библиотека, которую мы "создаём на продажу", будет называться `Differ` . Мне сейчас уже нужно куда-то запихнуть это название? Как? Переименовать `App.java` в `Differ.java` ?

Или это потом, в следующих шагах создадим отдельный файл?

Решено

2

Показать ещё комментарии (4)

Maksim Litvinov **Поддержка** 24 августа 2022

Приветствую, Даниил! Как мы осуждали чуть выше, наше приложение состоит из двух частей. Первая - это сама библиотека с методом Differ.generate(), которая сравнивает два файла. Вторая - консольная утилита, которая позволяет взаимодействовать с библиотекой из командной строки. Разработкой библиотеки мы займемся в следующих шагах, а пока вам нужно сделать консольную утилиту с минимальным функционалом - она бутет пока только выводить справку. Библиотека picocli предназначена как раз для создания консольных утилит и позволяет очень здорово упростить процесс создания. Подключите библиотеку, изучите в документации пример готового приложения и напишите собственное, чтобы она пока только генерировала справку:

развернуть

+7 495 085 28 38
бесплатно по Москве

8 800 100 22 47
бесплатно по РФ

support@hexlet.io

Направления

Курсы «Backend-разработка»

Курсы «Frontend-разработка»

Курсы «Создание сайтов»

Курсы «Тестирование»

Курсы «Аналитика данных»

Интенсивные курсы

Курсы DevOps

Курсы «Веб-разработка»

Курсы «Математика для программистов»

Бесплатные курсы

Все курсы

Курсы JavaScript

Курсы Python

Курсы Java

Курсы PHP

Курсы Ruby

Курсы Go

Курсы HTML

Курсы SQL

Курсы Git

Корпоративное обучение

Все категории

О Хекслете

О нас

Карьера в Хекслете

Хекслет Колледж

Подарочные сертификаты

Пригласить друзей

Карта вакансий

Документы

Условия использования

Соглашение об обработке ПД

Оферта

Акции

Помощь

Справка

Вопросы и ответы

Сообщество

Дополнительно

EN RU KZ

ООО «Хекслет Рус», 108813, г. Москва,
вн.тер.г. поселение Московский, г. Московский, ул. Солнечная,
д. 3А, стр. 1, помещ. 10/3
ОГРН 1217300010476

© Хекслет, 2024